Projeto 1 - Busca e Organização de Dados

Estruturas de Dados em C

SCC0202 - Algoritmos e Estruturas de Dados I

Objetivo

Implementar, em C, um programa para **carregar, buscar e organizar registros** usando diferentes estruturas de dados. O usuário deverá escolher:

- a estrutura que armazenará **todo o dataset**;
- a estrutura que armazenará **os resultados da busca**.

Cada linha do arquivo representa um `ITEM`. O mesmo programa deverá ser utilizado nas duas partes do trabalho.

Parte	Aplicação	Valor
1	Busca e organização de registros de aproximação de asteroides	5,0
2	Reutilização da solução com um dataset público escolhido pelo grupo	5,0

Estruturas de dados

O programa deve oferecer as seguintes opções:

`pilha, fila, lista_seq, lista_enc, lista_cabeca, lista_ord, lista_gen, lista_cruz`

As estruturas devem ser implementadas pelo grupo por meio de TADs. Não é permitido substituir as implementações por contêineres prontos de bibliotecas externas.

Atenção às listas avançadas

- **Lista com cabeça**: deve possuir um nó cabeça/sentinela real, que não representa um registro. Inserção, busca, remoção e percurso devem utilizar essa representação.
- **Lista ordenada**: a ordem deve ser mantida durante a inserção. Não será permitido inserir tudo e ordenar ao final.
- **Lista generalizada**: deve permitir organizar itens em grupos e subgrupos. Por exemplo, na Parte 1, agrupar por `threat_category` e depois por `year`.
- **Lista cruzada**: o mesmo item deve participar de duas organizações simultâneas, sem duplicação do registro. Por exemplo, permitir percorrer itens por `threat_category` ou por `year`.

A implementação deve deixar claro que essas estruturas são diferentes entre si e que a escolha feita pelo usuário altera efetivamente a organização dos itens.

Execução do programa

Parte da correção será automatizada. O executável deve se chamar `trabalho` e seguir o padrão abaixo:

```
./trabalho \
  --reader nasa \
  --input dados/asteroids.csv \
  --base lista_enc \
  --result lista_ord \
  --query consultas/q1.txt \
  --output resultado.csv \
  --top-k 100 \
  --limit 10000
```

Argumento	Função
`-reader`	leitor do dataset utilizado
`-input`	arquivo de entrada
`-base`	estrutura usada para armazenar o dataset completo
`-result`	estrutura usada para armazenar o resultado da busca
`-query`	arquivo contendo os critérios da busca
`-output`	arquivo que receberá os registros encontrados
`-top-k`	máximo de registros exportados; `0` significa todos
`-limit`	máximo de registros carregados; `0` significa toda a base

A estrutura indicada por `-base` deve receber os registros durante a leitura. A busca deve percorrer essa estrutura. Os registros aceitos devem ser inseridos na estrutura indicada por `-result`.

Consulta e saída

A consulta será fornecida em arquivo texto. O formato abaixo deverá ser suportado:

```
FILTER|is_future_event|==|true
FILTER|distance_au|<=|0.08
SIM|velocity_km_s|15.0|10.0|1.0
SIM|risk_score|0.70|0.30|2.0
ACCEPT|0.60
ORDER|similarity|DESC
GROUP|threat_category|year
CROSS|threat_category|year
```

Os comandos têm as seguintes funções:

- `FILTER`: condição obrigatória para o registro ser aceito;
- `SIM`: critério numérico de similaridade, no formato `campo|alvo|tolerancia|peso`;
- `ACCEPT`: limiar mínimo para a similaridade total;
- `ORDER`: chave usada pela lista ordenada;
- `GROUP`: campos usados pela lista generalizada;
- `CROSS`: campos usados pela lista cruzada.

Para um valor x, alvo q e tolerância $t > 0$, pode-se usar:

$$s(x, q, t) = \max\left(0, 1 - \frac{|x - q|}{t}\right)$$

A similaridade total será a média ponderada dos critérios `SIM`. Se não houver `SIM`, os filtros são suficientes para definir os resultados.

Arquivo de saída

O arquivo indicado por `-output` deve manter **o mesmo formato do arquivo de entrada**: mesmo cabeçalho, mesmas colunas, mesma ordem das colunas e mesmo delimitador. Não devem ser acrescentadas colunas auxiliares ao resultado.

Na saída padrão, imprimir pelo menos:

```
STATUS=OK
INPUT_RECORDS=10000
MATCHED_RECORDS=487
OUTPUT_RECORDS=100
LOAD_MS=12.431
SEARCH_MS=3.288
RESULT_INSERT_MS=0.812
WRITE_MS=1.050
```

Parte 1 - Asteroides (5,0 pontos)

Motivação

Registros de aproximação de asteroides permitem realizar buscas por data, distância, velocidade, magnitude e indicadores de risco. O objetivo desta parte é usar um problema concreto para observar como diferentes estruturas afetam a inserção, a busca, o percurso, a ordenação e a obtenção dos resultados.

Dataset

O arquivo da Parte 1 será disponibilizado no **e-Disciplinas**. O grupo deve utilizar exatamente a versão fornecida para garantir compatibilidade com a correção automática.

Os principais campos são:

Campo	Descrição
`asteroid_designation`, `asteroid_fullname`	identificação do asteroide
`close_approach_date`, `year`, `month`	data da aproximação
`distance_au`	distância mínima da Terra
`velocity_km_s`	velocidade relativa
`absolute_magnitude`	magnitude absoluta
`days_until_approach`	diferença em dias para a aproximação
`is_past_event`, `is_future_event`	indicadores temporais
`risk_score`, `panic_level`	indicadores numéricos presentes no dataset
`threat_category`, `panic_verdict`	categoria e descrição textual
`on_sentry_list`	indicador de presença na lista Sentry
`sentry_impact_prob`, `sentry_torino_scale`, `sentry_palermo_scale`, `sentry_diameter_km`	informações Sentry quando disponíveis

Cada linha deve ser tratada como um `ITEM`. Valores ausentes devem ser tratados sem interromper a leitura.

O que deve funcionar

1. carregar toda a base na estrutura escolhida por `-base`;
2. executar a consulta percorrendo essa estrutura;
3. inserir os registros encontrados em uma outra estrutura, que será escolhida por `-result`;
4. aceitar qualquer combinação válida entre as estruturas;
5. salvar corretamente os primeiros `top-k` itens da estrutura de resultados, conforme regras de acesso à estrutura que foi utilizado (–result);
6. liberar toda a memória ao final da execução.

Para `lista_gen` e `lista_cruz`, os comandos `GROUP` e `CROSS` devem ser usados para construir a organização dos resultados.

Parte 2 - Dados Abertos (5,0 pontos)

Motivação

Dados públicos abertos permitem acompanhar gastos, contratos, serviços, indicadores e outras ações do poder público. Além de favorecer reutilização e pesquisa, esses dados apoiam transparência, controle e fiscalização cidadã. Nesta parte, o objetivo é mostrar que as estruturas implementadas são reutilizáveis e não dependem do domínio de asteroides.

O grupo deverá escolher um dataset em:

`https://dados.gov.br/home`

O tema é livre, desde que existam registros suficientes para buscas e organização por diferentes campos.

Atenção: um mesmo conjunto de dados não poderá ser utilizado por mais de um grupo. Há um formulário específico e planilha no e-disciplinas para definição dos grupos e definição do dataset.

Requisito principal

A Parte 2 deve reutilizar **as mesmas implementações** de pilha, fila e listas. Apenas o módulo responsável por transformar uma linha do novo dataset em um `ITEM` poderá ser específico da base escolhida.

Em outras palavras, o cliente deve poder trocar o leitor, por exemplo:

```
--reader nasa
--reader compras
--reader contratos
```

sem criar novas versões das estruturas de dados.

O grupo deve definir pelo menos duas consultas úteis para o dataset escolhido e demonstrar o uso de pelo menos uma das estruturas avançadas: `lista_cabeca`, `lista_gen` ou `lista_cruz`.

Análise de complexidade e avaliação experimental

O relatório deve apresentar a complexidade assintótica das operações realmente utilizadas.

Estrutura	Inserção	Busca	Remoção	Observações
pilha				
fila				
lista sequencial				
lista encadeada				
lista com cabeça				
lista ordenada				
lista generalizada				
lista cruzada				

Também deverá ser realizada uma avaliação de tempo variando progressivamente o tamanho da entrada e o valor de `top-k`. Cada configuração deve ser executada pelo menos três vezes e deve ser apresentada a média.

N	Base	Resultado	k	Carga	Busca	Inserção res.	Saída	Total
1.000			10					
5.000			50					
10.000			100					
25.000			500					
50.000			1000					
toda a base			todos					

Os valores podem ser ajustados ao tamanho do dataset. A discussão deve comparar os tempos observados com a complexidade esperada das estruturas.

Critérios de avaliação

Critério	Valor
Correção funcional, entrada, saída e compatibilidade com os testes automáticos	1,5
Uso correto de TADs e separação entre dados e estruturas	2,0
Implementação das estruturas de dados, com atenção especial a lista com cabeça, generalizada e cruzada	2,5
Qualidade do código, modularização, legibilidade e comentários relevantes	1,0
Análise de complexidade assintótica	1,5
Avaliação experimental de tempo e discussão dos resultados	1,5
Total	**10,0**

Entrega

A entrega deverá conter os seguintes itens:

1. **Código-fonte completo do projeto**, incluindo os arquivos de implementação, headers e `Makefile`. O comando `make` deve gerar o executável `trabalho`.

2. **Relatório do projeto**, descrevendo de forma breve a organização do código, os principais módulos e as estruturas de dados implementadas. O relatório deve incluir **seis exemplos completos de execução**: três referentes à Parte 1 e três referentes à Parte 2. Para cada exemplo, devem ser apresentados o comando utilizado, a consulta realizada e o resultado obtido.

3. **Relatório experimental**, contendo a análise da complexidade assintótica das operações utilizadas e a avaliação experimental do custo computacional. Devem ser apresentados os quadros de tempos, as configurações utilizadas nos experimentos e uma discussão comparando os resultados observados com os custos assintóticos esperados.

O código deve compilar com `-Wall -Wextra -pedantic`. A execução usada na correção não poderá depender de menus ou interface gráfica. Recursos interativos podem existir, mas o padrão de argumentos deste documento deve funcionar integralmente.

Atenção: Em relação à parte 2 do projeto, o grupo poderá, opcionalmente, apresentar o trabalho na Feira de Extensão do ICMC/USP (Novembro). É necessário preencher o formulário específico no e-disciplinas.